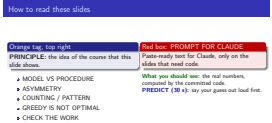
The Shortest-Path Problem

A correct procedure on the wrong model is still wrong

COMSC 2043 • Discrete Structures and Critical Thinking

Week 12, Thursday, November 5, 2026

Thursday, 75 minutes. This is the one story Jeremy has told for years: a one-way street, a six-sided die, and a tour of six cities. Everything today is that story. The sentence to land by the end: a correct procedure on the wrong model is still wrong. Say it now once, then let the story earn it.



Walk the legend in thirty seconds. Orange tag means this slide demonstrates one of five course ideas; the index at the end lists them all. Red box means code and a paste-ready prompt. Green line means: here is what the screen shows; those numbers are computed by committed code, not by hand. Question for the room: which of the five do you already think the one-way street will show? Expected: asymmetry, maybe model vs procedure.

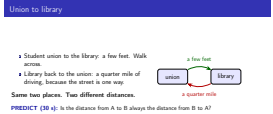
- A one-way street (the hook)
- A six-sided die gives six cities, and a table of distances
- A tour: 1-2-3-4-5-6-1, and what we are optimizing
- Greedy: always go to the nearest city, and where it is fooled
- Guess and check with the die
- How many different tours? A pattern, and a plot
- Then we started writing code
- Close: model versus procedure, and the Gate

Read the eight steps. Tell them: you will not write code until step seven, because the model comes first. That is Tuesday's point and today's. Question: why do you think the code comes last? Expected: because we need to know what we are computing before we compute it.

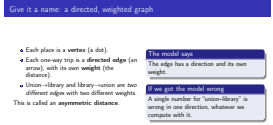
Words we will use

graph	Places (dots) and links between them (arrows or lines).
directed edge	A link with a direction: it goes from one place to another.
weight	A number on a link, here a distance.
tour	A trip that visits every city once and comes back to the start.
greedy	Always take the cheapest-looking next step.
brute force	Try every possibility and keep the best.
factorial ($n!$)	$n \times (n-1) \times \cdots \times 1$. For example $4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$.
permutation	One ordering of a list of things.
test (assert)	A line of code that stops with an error if a claim is false.

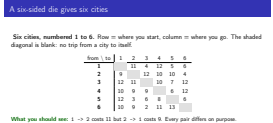
Do not read the table. Say: if a word on a later slide is not clear, it is here. Point out factorial: 4 factorial is 4 times 3 times 2 times 1. We will use it at slide on counting. Ask: which of these words have you used already? Expected: graph, test; the rest are new today.



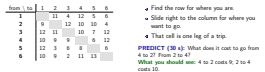
Tell the story as you have told it for years, in your own words. Ask the room first: how far is it from the union to the library? Then: how far from the library back to the union? Let someone say it is the same, then break it with the one-way street. Expected answers: a few feet one way; the long way around the other. Land it: the distance depends on the direction. We usually pretend it does not.



Name what they just said. Dots are vertices, arrows are directed edges, the number on an arrow is its weight. Say plainly: this is a modeling choice, not a fact of nature. Question: if I only wrote one number on a line between union and library, what would I be assuming? Expected: that the trip is the same both ways. Tie to Tuesday: that unstated assumption is the drift we checked for in the A to D activity.



This table is fixed and hand-checkable; it lives in data/distances.csv and every number in this deck comes from it. Students may roll their own: roll a die twice for each cell, or use a number from one to twelve. Their answers will differ from ours, and that is fine because the method is the same. Ask: find me a pair where the two directions are equal. Expected: there is none, I made it so. Read one row: from city 1, going to 2 costs 11, to 3 costs 4.

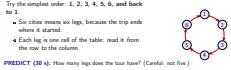


from \ to	1	2	3	4	5	6
1		11	4	12	5	9
2	9		12	10	10	4
3	12	11		10	7	12
4	10	9	9		6	12
5	12	3	6	8		6
6	10	9	2	11	11	

- Find the row for where you are.
- Slide right to the column for where you want to go.
- That cell is one leg of a trip.

PREDICT (30 s): What does it cost to go from 4 to 2? From 2 to 4?
What you should see: 4 to 2 costs 9; 2 to 4 costs 10.

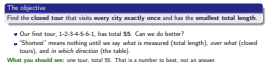
Make them do it, not you. Row first, then column. Ask for 4 to 2 and for 2 to 4; the two answers are 9 and 10, which also shows asymmetry on a pair that is close but not equal. Common mistake: reading column then row, which silently reverses every leg. That is a model error with a correct-looking procedure.



Draw six dots on the board and walk 1 to 2 to 3 and so on, ending with the arrow back to 1. Ask for the number of legs. Expected: six; the common answer is five because we visited six cities, but the trip back home is a leg too. The picture is the model, the table is the data.



Do the addition on the board one leg at a time, with the highlighted cells showing where each number came from. Hand work: eleven plus twelve is twenty-three, plus ten is thirty-three, plus six is thirty-nine, plus six is forty-five, plus ten is fifty-five. Ask: what is the return leg, and did you remember it? Expected: 6 to 1, which is ten. The reverse tour 1-6-5-4-3-2-1 uses six different cells; its total is 56, so it is a different tour, not the same one backwards.



This is Tuesday's line: shortest needs an objective and weights. Say all three out loud: what is measured, over which choices, in which direction. Ask: is fifty-five the shortest tour? Expected: we do not know, we have only tried one. Setup for the next three sections: three ways to look for a better tour.

Idea 1: greedy, always go to the nearest city

Rule: from where you are, look at the cities you have not visited and go to the one with the smallest distance. Ties: lowest city number.

Start at city 1.

- From 1: options 2 (11), 3 (4), 4, 5 (5), 6. Nearest is 3 at 4.

PREDICT (30 s): From 3, which unvisited city is nearest?

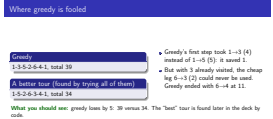
Work it with the table on the board. From 1 the row reads 11, 4, 12, 5, 6, so 3 wins at 4; 5 is close behind at 5. Remember that: it matters in two slides. Ask them to find the nearest from 3 among 2, 4, 5, 6. Expected: 5, at 7.

from \ to	1	2	3	4	5	6
1		11	**4**	12	5	9
2	9		12	10	10	**4**
3	12	11		10	**7**	12
4	**10**	9	9		6	12
5	12	**3**	6	8		6
6	10	9	2	**11**	13	

1→3	4
3→5	7
5→2	3
2→6	4
6→4	11 (only 4 is left)
4→1	10 (the way home)

What you should see: tour **1-3-5-2-6-4-1**, total **39**. Better than 55.

Follow the highlighted cells. At each stop say the choices and pick the smallest. At 6 only city 4 is left, so there is no choice; the cheap-looking path has run out of cities. Total: four plus seven plus three plus four plus eleven plus ten is 39. Ask: is thirty-nine the best possible? Expected: it beat fifty-five, but we have no reason yet to believe it is the best.



This is the heart of greedy-is-not-optimal. A cheap move now closes off a cheap leg later. Show the numbers: greedy saved one at the first step and was then forced into an eleven-long leg, 6 to 4, at the end. Procedure was followed perfectly; what failed is the assumption that the locally cheapest step leads to a globally cheapest tour. Ask: what did greedy assume? Expected: that the best next step is part of the best whole tour. We will check the best tour by code; for now accept it.

Start at city 1. Roll the die for the next city.

- If the city is already in the tour, **reject** the roll and roll again.
- When all six cities are in, add the way home, total the six legs, and write it down.
- Keep the **best so far**. Repeat.

PREDICT (30 s): Is a rejected roll free?

Roll a real die if you have one. The rejection rule matters: a rejected roll costs time but does not change the tour. Ask whether rejections are free. Expected: not free in time; the die has no memory, so late in the tour most rolls are rejected. Next slide puts a number on it.

- Slot 2: five of six faces work. Slot 3: four of six. … Slot 6: only one face works.
- With k cities still open out of 6, you need $6/k$ rolls on average.
- Over five slots: $6/5 + 6/4 + 6/3 + 6/2 + 6/1$.

What you should see: about **13.7** rolls to fill one tour, not 5.

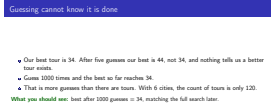
The last city is the painful one: you wait for one particular face out of six, six rolls on average. Add the five fractions on the board: one point two, one point five, two, three, six. Total 13.7. Common mistake: saying five rolls per tour. The code section does the same thing: it rolls among all six and rejects repeats.

A short worked trace

#	rolls for slots 2–6	tour	total	best so far
1	2 3 5 4 4 6	1-2-3-5-4-6-1	60	60
2	1 2 5 6 3 2 6	1-3-2-5-6-4-1	44	44
3	5 2 5 6 4 3	1-5-2-6-4-3-1	44	44
4	6 6 5 2 4 3	1-6-5-2-4-3-1	53	44
5	2 4 2 5 2 1 1 2 2 6	1-2-4-5-3-6-1	55	44

What you should see: best after 1 guess: 60. After 3: 44. After 5: 44. It only ever goes down or stays.

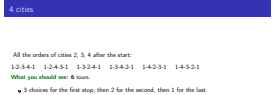
Walk row one: rolls two, three, five, four, then a four that is rejected because 4 is already in, then six. Tour 1-2-3-5-4-6. Total it with the table. Best so far starts equal to the first total. Key line: the best so far can never rise; it only drops when we luck into something better. This trace uses a fixed random seed so it is reproducible; yours will differ. Ask: after five guesses, are we sure of the best? Expected: no.



Guess and check is honest but blind. It never knows it is finished. The thousand-guess number comes from the same code; it equals the true best only because six cities have so few tours. Foreshadow: with fifteen cities guessing a thousand times would be a tiny sample. Question: how could we be sure of the best? Expected: look at every tour. How many are there? That is the next section.



Build this on the board, not from the slide. For two cities there is one tour: go to the other and come back. For three, after 1 you can choose 2 or 3, and then the last is forced. Notice that 1-2-3-1 and 1-3-2-1 are different tours because this table is asymmetric. Ask for guesses for four cities. Expected guesses: 3, 4, 6, 8. Collect them without comment.



Let students call out the list while you write it; the code that made this list is in the last section. Compare with their guesses. Point at the pattern: three choices, then two, then one. Do not give the formula yet. Question: what would five cities look like? Expected: four choices, then three, two, one.

Choices shrink by one each step

For n cities with the start fixed, there are $n-1$ cities left to order:

slot	2nd stop	3rd	4th	5th	6th
choices (6 cities)	5	4	3	2	1

Multiply: $5 \times 4 \times 3 \times 2 \times 1 = 5!$ $\Rightarrow$ $(n-1)!$ tours.

What you should see: 24 tours for 5 cities, 120 tours for 6.

The reasoning is the point, not the numbers. Each time you pick, one fewer city remains. The product of a shrinking list is a factorial. Ask: why does the first slot have n minus 1 choices and not n? Expected: because the start city is already used. Check: 5 times 4 is 20, times 3 is 60, times 2 is 120, times 1 is 120.

cities n	2	3	4	5	6
tours $(n-1)!$	1	2	6	24	120

PREDICT (30 s): What is the next number, for 7 cities?

- Each step multiplies by the next whole number: $\times 2$, $\times 3$, $\times 4$, $\times 5$.

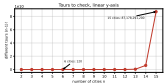
Let the room say the pattern: each new city multiplies the count by n minus 1. Ask for 7 cities; it is 6 times 120, so 720. (That is the computer's number too: the code section asserts the formula for n from 2 to 6.) Say: the numbers look small now. Keep that feeling for two slides.

- **Our convention:** start fixed at city 1, direction matters. Count $= (n-1)!$.
- Do not fix the start: list all $n!$ orders, but each tour appears n times (rotations), and you get the same $(n-1)!$ tours. For 6 cities: 720 listings, 120 tours.
- Ignore direction (a two-way street): a tour and its reverse are the same, so halve: $(n-1)!/2$, which is 60 for 6 cities.

What you should see: Our table is asymmetric, so a tour and its reverse are different tours. **No halving.**

Say this once and move on. If you do not fix the start, you can list six factorial, seven hundred twenty, ordered lists of six cities, but they describe the same tours six times each. If the street were two-way and the distances symmetric, a tour and its reverse would have the same total, and you could halve the count. Our table is one-way on purpose, so we do not. Jeremy's convention is the first bullet. Ask: which convention does the word asymmetric rule out? Expected: halving.

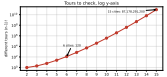
Plot it (linear)



What you should see: the curve is flat for a few cities, then it shoots off the page.

Drawn by `scripts/make_figure.py`. Ask what they see. Expected: almost nothing happens until about ten, then it explodes. The linear picture hides how fast it grows at the start, which is why we need the log version next.

Plot it (log)



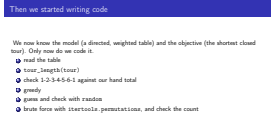
What you should see: each step up the axis is a factor of ten; the line still bends upward.

On a log axis an exponential becomes a straight line; this bends upward, so it grows faster than exponential. Ask them to read the 15-city point. Do not rush it. Ten cities is 362 880 tours; fifteen is 87 178 291 200.

- 6 cities: **120** tours. A laptop does that instantly.
- 10 cities: 362 880 tours.
- 15 cities: **87 178 291 200** tours (about 87 billion).

What you should see: at one million tours per second, 15 cities takes about 24.2 hours. (The speed is an assumption.)

Trying every tour is correct, because it checks everything, and hopeless for large n. Correct and not practical are different things. At one million tours per second, a made-up but generous speed, fifteen cities takes about a day. Ask: what do we do instead of brute force? Expected: smarter methods, or accept a good-enough answer such as greedy. That is why people care about the shortest-path problem.



Say the order out loud: model first, then procedure. Each step has a prompt for Claude, and each prompt asks Claude to show its work so you can compare with the hand work on the earlier slides. Predict-run-explain: have them guess the output before running. All the reference code is in code/tour.py and the tests are in `code/test_tour.py`. If Claude's version differs in style, that is fine; compare the numbers.

Step 1: read the table

PROMPT FOR CLAUDE — prompts/01_read_table.md

```
Make a folder called tour_demo. Put data/distances.csv in it (I will paste it).
Write tour.py with a function read_table(path) that reads the CSV and returns a dictionary
keyed by (from, to) pairs, using their numbers. The diagonal cells are blanks, so skip them.
Do not add anything else yet.
Then add a few lines at the bottom (under an if __name__ == "__main__": block) that print
the distance from 1 to 2 and from 2 to 1. Run it and show me the output.
```

What you should see: 11 for 1 to 2, and 9 for 2 to 1: the two directions differ.
PREDICT (30 s): Will the two printed numbers be equal?

Paste, run, read the output aloud. The point is the data structure: a dictionary keyed by the pair (from, to), so (1, 2) and (2, 1) are separate entries. That is the model written in code. If Claude uses a list of lists, ask it to switch to a dictionary keyed by pairs. Recovery: the reference is code/tour.py.

PROMPT FOR CLAUDE prompts/02_tour_length.md

```
In tour.py add tour_length(tour, dist): tour is a list of city numbers like [1, 3, 5, 4, 6, 2].
It must add up every leg, INCLUDING the leg from the last city back to the first.
Add a second function legs(tour, dist) that returns each leg as (from_city, to_city, distance)
so I can compare it with my hand work. Keep both functions short. Do not change read_table.
```

What you should see: legs gives six triples; tour_length adds their distances.
PREDICT (30 s): What goes wrong if the code forgets the leg from the last city back to the first?

Predict answer: the total comes out short by one leg (the way home). Show that by hand. Explain beat: closed tour means the return leg counts. Common mistake: range over five legs not six.

Step 3: check our hand total

PROMPT FOR CLAUDE prompts/03_check_hand.md

```
In tour_demo, run legs([1, 2, 3, 5, 8]) and test_length on the same tour.
Print the six legs and the total. I will compare them with the legs and total I added up by hand.
If they differ, do not change my numbers: tell me which leg differs and why.
```

What you should see: legs 11, 12, 10, 6, 6, 10; total **55**, the same as our hand total.

Compare with the hand total from the tour slide. If they differ, the code or the hand work is wrong; find which. Do not let Claude adjust the expected number until they agree. This is checking a procedure against a hand check, which is Tuesday's move.

Step 3b: tests guard the model

PROMPT FOR CLAUDE prompts/03_check_tour.md

```
Create test_tour.py with pytest tests using read_table("data/distances.csv").
1. test_symmetry: for every (a, b), tour_length([a, b]) == tour_length([b, a]).
2. test_length: tour_length([1, 2, 3, 4, 5, 6]) equals the total I give you (I will paste my hand total).
Run pytest and show me the one-line summary.
```

What you should see: 2 passed. Test 1 fails if someone "fixes" the table to be symmetric.

This test is the model written as a check. If somebody makes the table symmetric, the test fails and tells you the model has changed. Ask: which test protects the model and which protects the procedure? Expected: the first protects the model; the second protects the total.

PROMPT FOR CLAUDE prompts/08_greedy.md

```
In tour.py add greedy_tour(dist, start=1). From the current city, go to the unvisited city
with the smallest distance. If two cities tie, take the lower city number. Return the tour as a list.
Run it from city 1, print the tour and tour_length. Show me each step: current city, the options
with their distances, and the one it chose, so I can check it against my hand work.
```

What you should see: tour 1-3-5-2-6-4-1, total **39**, matching the hand work.
PREDICT (30 s): Which city does greedy choose from city 6?

Predict answer: city 4, the only one left, with a leg of 11. Have them compare each printed step with the highlighted table on the greedy slide. Tie rule matters when two options are equal; our table has no ties from city 1, but the code must still say what to do.

PROMPT FOR CLAUDE prompts/05_guess_and_check.md

```
In tour.py add guess_tour(dist, rng, start=0): keep the start city, then pick a random city from
all six with rng.choice(), if it is already in the tour, reject it and pick again; return the tour
and its total. Add a function guess_and_check(dist, k, seed): make k tours with random.Random(seed), and for each print
the evila, the tour, its total, and the best total so far. Run it with k=5 and seed=3114 and show me the output.
```

What you should see: with seed 3114: best after 1 guess 60, after 3 44, after 5 44; same as the trace slide.

Using the seed is what makes the run repeat: same seed, same rolls. Students who change the seed get different tours, and that is the point of guessing. Ask: why does the best so far never go up? Expected: it keeps the minimum of everything seen.

PROMPT FOR CLAUDE prompts/06_brute_force.md

In tour.py add brute_force(dist, start=1) using itertools.permutations on the other cities, so the full tour is start + a permutation + start. Return (best_tour, best_total, num_of_tours_tried). Run it and show me all three values.

What you should see: best tour 1-5-2-4-3-1, total **34**, 120 tours tried.
PREDICT (30 s): Will brute force beat greedy, tie it, or lose to it?

Predict answer: it can never lose, since greedy's tour is one of the tours brute force tries. Here it wins by 5. The count 120 is the formula from the counting slides.

Step 6b: assert the count

PROMPT FOR CLAUDE prompts/06_brute_force.md

```
Add two pytest tests to test_tour.py:
1. for n = 2 to 6 (use only the first n cities of the table), the count brute_force returns equals
   (n-1)!/2 (use math.factorial).
2. brute_force's best total is less than or equal to the total of greedy_tour.
Run pytest and show me the one-line summary.
```

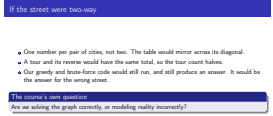
What you should see: pytest: passes: counts 1, 2, 6, 24, 120 for 2 to 6 cities, and brute force is no worse than greedy.

This closes the loop between the picture, the formula and the code. The formula predicted the count; the code counted it; the assert makes them agree forever. Reference result in this repo: all eight pytest tests pass (see REPORT.md for the exact line).

Model versus procedure: each step, again

Step	What it assumed	What would change
the table	the street is one way: each direction has its own weight	two-way: one number per pair, tours and reverses tie
the tour	visit every city once and come home	drop "come home": an open path, a different problem
greedy	the cheapest next step belongs to the cheapest tour	nothing in the table promises that
guess and check	random tours sample the space well	sample is tiny when the count explodes
brute force	the count is small enough to try all	fine at 6 cities, hopeless at 15

Go row by row and ask the room for the second column before showing it. The sentence to land: every procedure we ran was correct for its own model. The question is whether the model matches the street. If the street were two-way, the table would be symmetric, all distances equal both ways, and a tour and its reverse would have the same total; the count would be halved. Ask: which of these assumptions is easiest to forget? Expected: direction.



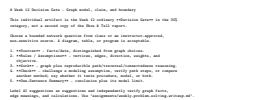
Read the block aloud; it is Week 12's own question. A program that runs without error says nothing about whether the model is right. That is the point of the AI failure we inspect: a correct answer to the wrong graph. Ask: if Claude gives you a shortest path, what is the first thing you check? Expected: what an edge means and whether it has a direction.

Where this came from

Jeremy first worked this as the shortest path problem in Python in 2020 to 2021: brute force, greedy, then ant colony ideas and MPI on a cluster.

Optional slide. Say it in one breath and move on; it is only a word of provenance. The brute force and greedy you saw are the first two steps of that path; ant colony and cluster computing are what you do when the count explodes.

Gate preview



```
# Week 12 Decision Gate — Graph model, claim, and boundary

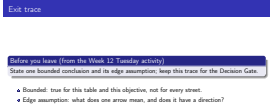
This individual artifact is the Week 12 ordinary reflection Gate== in the ME
category, not a second copy of the Show & Tell report.

Choose a bounded network question from class or an instructor-approved,
non-sensitive source. A diagram, table, or program is acceptable.

1. ==Source== — facts/data, distinguished from graph choices.
2. ==Rules / Assumptions== — vertices, edges, direction, weights, and
   boundaries.
3. ==Work== — graph plus reproducible path/traversal/connectedness reasoning.
4. ==Check== — challenge a modeling assumption, verify path steps, or compare
   two representations (e.g., adjacency matrix vs. adjacency list).
5. ==One-Sentence Summary== — conclusion plus its model limit.

Label AI suggestions as suggestions and independently verify graph facts,
edge meanings, and calculations. Use `assignments/weekly-problem-solving.reviewq.md`.
```

Read the heading and the five numbered steps. Tell them: your Gate is a graph model of your own bounded question; the five parts are the same five moves we made today: sources, assumptions, work, check, summary. This is the individual Decision Gate, not the Show and Tell report. Verbatim from week-12/student/decision-gate.md.



Collect one sentence from each student. Good example: for this table, greedy gives thirty-nine and brute force gives thirty-four, assuming one-way edges with the weights in the table. Reward narrowing a claim after critique, as the run-of-show says. Note: the Thursday run-of-show lists the exit trace under Tuesday's collect-a-bounded-conclusion move, so the wording here is taken from the Tuesday activity.

Slide	Title	Principle	Prompt
** 5	Union to library	ASYMMETRY	
6	Give it a name: a directed, weighted graph	MODEL VS PROCEDURE	
7	A no-sided die goes no silos.	ASYMMETRY	
8	A taxi, then buses	MODEL VS PROCEDURE	
10	Dijkstra: the idea, step by step	MODEL VS PROCEDURE	
11	What are we optimizing?	MODEL VS PROCEDURE	
12	Idea 1: greedy, always go to the nearest city	GREEDY IS NOT OPTIMAL	
13	Greedy, worked on the table from city 1	GREEDY IS NOT OPTIMAL	
14	Where greedy is fooled	GREEDY IS NOT OPTIMAL	

Use this to find a slide fast: slide number, which lesson idea it shows, and which prompt file it pairs with (prompts/NN, a = first block, b = second block).

Slide	Title	Principle	Prompt
15	Idea 2: guess and check with the die	CHECK THE WORK	
16	What repeating repeats roots	COUNTING / PATTERN	
17	A short worked trace	CHECK THE WORK	
18	Guessing cannot know it is done	CHECK THE WORK	
19	Count the teams: 2 cities, then 3	COUNTING / PATTERN	
20	4 outcomes	COUNTING / PATTERN	
21	Chosen shrink by one each step	COUNTING / PATTERN	
22	The pattern	COUNTING / PATTERN	
23	Which counting convention? Say it once	COUNTING / PATTERN	

Use this to find a slide fast: slide number, which lesson idea it shows, and which prompt file it pairs with (prompts/NN, a = first block, b = second block).

Slide	Title	Principle	Prompt
24	Plot it (linear)	COUNTING / PATTERN	
25	Plot it (log)	COUNTING / PATTERN	
26	What the curve means for brute force	COUNTING / PATTERN	
27	Then we started writing code	MODEL VS PROCEDURE	
28	Step 1: the length of a run	CHECK THE WORKING	01a
29	Step 2: the length of a run	CHECK THE WORKING	02a
30	Step 3: check our hand total	CHECK THE WORK	03a
31	Step 5b: trees guard the model	ASYMMETRY	03b
32	Step 6: greedy	GREEDY IS NOT OPTIMAL	06a

Use this to find a slide fast: slide number, which lesson idea it shows, and which prompt file it pairs with (prompts/NN, a = first block, b = second block).

Slide	Title	Principle	Prompt
33	Step 5: guess and check with random	CHECK THE WORK	05a
34	Step 5: brute force over permutations	COUNTING / PATTERN	05a
35	Step 6b: assert the count	COUNTING / PATTERN	05b
36	Step 6a: a procedure: each step, again	MODEL VS PROCEDURE	
37	If the stones were two-way	MODEL VS PROCEDURE	
38	Gate premise	CHECK THE WORK	
40	Exit trace	CHECK THE WORK	

Use this to find a slide fast: slide number, which lesson idea it shows, and which prompt file it pairs with (prompts/NN, a = first block, b = second block).